

数据库 (Database)

- 1. 第一章：引言 (Chapter 1: Introduction)
 - 1.1. 数据库系统概述 (Overview of Database Systems)
 - 1.2. 抽象级别 (Levels of Abstraction)
 - 1.3. 模式和实例 (Schemas and Instances)
 - 1.4. 数据模型 (Data Models)
 - 1.5. 数据库语言 (Database Languages)
 - 1.6. 事务管理 (Transaction Management)
- 2. 第二章：关系模型 (Chapter 2: The Relational Model)
 - 2.1. 关系数据库结构 (Structure of Relational Databases)
 - 2.2. 码 (Keys)
 - 2.3. 关系代数 (Relational Algebra)
 - 2.3.1. 基本运算 (Fundamental Operations)
 - 2.3.2. 附加及扩展运算 (Additional & Extended Operations)
- 3. 第三章：SQL (Chapter 3: SQL)
 - 3.1. 数据定义语言 (Data Definition Language - DDL)
 - 3.2. 基本查询结构 (Basic Query Structure)
 - 3.3. 附加的基本运算
 - 3.4. 聚合函数 (Aggregate Functions)
 - 3.5. 嵌套子查询 (Nested Subqueries)
- 4. 第四章：中级 SQL (Chapter 4: Intermediate SQL)
 - 4.1. 连接表达式 (Join Expressions)
 - 4.2. 视图 (Views)
 - 4.3. 事务 (Transactions)
 - 4.4. 完整性约束 (Integrity Constraints)
 - 4.5. 授权与安全 (Authorization and Security)
- 5. 第六章：关系数据库设计 (Chapter 6: Relational Database Design)
 - 5.1. 优良设计的特点 (Features of Good Relational Design)
 - 5.2. 函数依赖 (Functional Dependencies - FDs)
 - 5.3. 范式 (Normal Forms)
 - 5.4. BCNF 与 3NF 的比较 (Comparison of BCNF and 3NF)
- 6. 第七章：存储与文件结构 (Chapter 7: Storage and File Structure)
 - 6.1. 物理存储介质 (Physical Storage Media)

- 6.2. 文件中的记录组织 (Organization of Records in Files)
- 6.3. 文件组织方式 (File Organization)
- 7. 第八章：索引与散列 (Chapter 8: Indexing and Hashing)
 - 7.1. 索引的基本概念 (Basic Concepts of Indexing)
 - 7.2. B+ 树索引 (B+-Tree Index Files)
 - 7.3. 散列索引 (Hash Indices)
 - 7.4. 索引选择的权衡 (Trade-offs in Index Selection)
- 8. 第九章：查询处理 (Chapter 9: Query Processing) 仅作补充
 - 8.1. 查询处理概述 (Overview of Query Processing)
 - 8.2. 查询优化 (Query Optimization)
 - 8.3. 算法选择 (Algorithm Selection)
 - 8.4. 表达式求值 (Evaluation of Expressions)
- 9. 第十章：事务 (Chapter 10: Transaction)
 - 9.1. 事务的概念 (Transaction Concept)
 - 9.2. ACID 特性 (The ACID Properties)
 - 9.3. 事务的状态 (Transaction State)

1. 第一章：引言 (Chapter 1: Introduction)

本章我们将了解数据库系统的基本概念，以及为什么它们在当今世界如此重要。

1.1. 数据库系统概述 (Overview of Database Systems)

首先，我们来弄清楚两个核心概念：**数据库**和**数据库管理系统**。

- **数据库 (Database)**：可以看作是一个**相互关联的数据的集合**。它存储了关于特定企业或领域的信息。
 - 例如：一个大学的数据库可能包含关于学生、课程、教师等方面的信息。
- **数据库管理系统 (Database Management System - DBMS)**：是一个用于**创建和管理数据库**的软件系统。它为用户提供了一个方便、高效的环境来存取和操作数据库中的数据。
- **数据库系统的目的 (Purpose of Database Systems)**：与早期**文件处理系统 (File-Processing Systems)** 相比，数据库系统解决了许多问题，例如：
 - **数据冗余和不一致 (Data redundancy and inconsistency)**：相同的信息可能被存储在多个文件中，更新不及时会导致数据不一致。
 - **数据访问困难 (Difficulty in accessing data)**：需要为每个新任务编写新的程序来提取数据。
 - **数据孤立 (Data isolation)**：数据分散在不同格式的文件中，难以整合。

- **完整性问题 (Integrity problems)**: 难以在程序中添加新的约束条件 (例如, 账户余额不能为负)。
- **原子性问题 (Atomicity problems)**: 系统故障可能导致操作只完成了一部分, 使数据处于不一致状态 (例如, 转账只扣款未存款)。
- **并发访问异常 (Concurrent access anomalies)**: 多用户同时访问可能导致不一致的结果。
- **安全性问题 (Security problems)**: 难以控制不同用户对数据的访问权限。

1.2. 抽象级别 (Levels of Abstraction)

为了使系统对用户更友好, DBMS 隐藏了数据存储和维护的复杂细节, 这通过三个抽象级别实现:

- **物理层 (Physical Level)**: 最低级别, 描述了**数据究竟是如何存储的 (how data is stored)**。这包括复杂的数据结构细节。
- **逻辑层 (Logical Level)**: 中间级别, 描述了**数据库中存储了什么数据以及这些数据之间存在什么关系 (what data is stored, and what relationships exist)**。这是数据库管理员 (DBA) 使用的层面。
- **视图层 (View Level)**: 最高级别, 只描述整个数据库的一部分 (**a part of the entire database**)。它为不同用户提供不同的视图, 隐藏了数据库的其余部分, 简化了交互并提供了安全保障。

1.3. 模式和实例 (Schemas and Instances)

- **模式 (Schema)**: 指数据库的**逻辑结构 (the logical structure)**, 它定义了所有数据及其关系。可以看作是数据库的设计蓝图。物理模式对应物理层, 逻辑模式对应逻辑层。
- **实例 (Instance)**: 指在**某一特定时刻, 存储在数据库中的信息的集合 (the collection of information stored in the database at a particular moment)**。数据库的实例会随着数据的增删改而不断变化。

1.4. 数据模型 (Data Models)

数据模型是一个描述数据、数据关系、数据语义和一致性约束的工具集合。

- **关系模型 (Relational Model)**: 使用**表格 (tables)** 的集合来表示数据和关系。这是最主流的数据模型。
- **实体-关系模型 (Entity-Relationship Model - E-R Model)**: 主要用于数据库设计。它将真实世界看作是由**实体 (entities)** 和实体间的**关系 (relationships)** 构成的。
- **其他模型 (Other Models)**: 还包括面向对象模型 (Object-based model)、半结构化模型 (Semistructured model) 等。

1.5. 数据库语言 (Database Languages)

- **数据定义语言 (Data Definition Language - DDL)**: 用于定义数据库模式。

- 例如：CREATE TABLE 语句用于创建一个新的表。
- **数据操纵语言 (Data Manipulation Language - DML)**：用于访问和操作数据。
 - 主要包括：查询 (retrieval)、插入 (insertion)、删除 (deletion)、修改 (modification)。
 - **SQL (Structured Query Language)** 是最广泛使用的数据库语言，它同时包含了 DDL 和 DML 的功能。

1.6. 事务管理 (Transaction Management)

事务 (Transaction) 是数据库应用中执行工作的一个逻辑单元，它通常包含一系列的操作。

- 为了维护数据在系统故障或并发访问情况下的一致性 (consistency)，数据库系统需要确保事务具有 **ACID** 特性：
 - **原子性 (Atomicity)**：事务中的所有操作要么**全部执行**，要么**全不执行**。
 - **一致性 (Consistency)**：事务的执行必须使数据库从一个**一致状态**转变到另一个**一致状态**。
 - **隔离性 (Isolation)**：并发执行的多个事务之间应**相互隔离**，一个事务的执行不应被其他事务干扰。
 - **持久性 (Durability)**：一个已成功提交的事务对数据库的改变必须是**永久的**，即使后续发生系统故障。

2. 第二章：关系模型 (Chapter 2: The Relational Model)

2.1. 关系数据库结构 (Structure of Relational Databases)

关系模型用一种非常直观的方式——**表格**——来表示数据。

- **基本概念 (Basic Concepts)**
 - **关系 (Relation)**：本质上就是一张**二维表**。表中的行和列分别对应记录和字段。
 - **元组 (Tuple)**：表中的一行，代表一个实体或一条记录 (record)。
 - **属性 (Attribute)**：表中的一列，代表实体的一个特征。
- **属性类型 (Attribute Types)**
 - 每个属性都有一个允许取值的集合，称为**域 (Domain)**。例如，student_id 的域可以是所有6位数字组成的字符串。
 - **Null 值**：是一个特殊的标记，表示值**未知 (unknown)** 或**不存在 (does not exist)**。
- **关系模式与实例 (Relation Schema and Instance)**
 - **关系模式 (Relation Schema)**：描述了一个关系的“框架”，包括**关系名**和其**属性列表**。例如：instructor(ID, name, dept_name, salary)。

- **关系实例 (Relation Instance)**: 在特定时刻, 该关系模式下的具体数据。也就是带数据的表格本身。

2.2. 码 (Keys)

码 (Key) 是一个或多个属性的集合, 用于在关系中唯一地标识元组。

- **超码 (Superkey)**: 一个或多个属性的集合, 其值可以**唯一确定**关系中的一个元组。例如, 在学生表中, $\{ID\}$ 是一个超码, $\{ID, name\}$ 也是一个超码。
- **候选码 (Candidate Key)**: 是**最小的超码**。即它的任何真子集都不能成为超码。例如, $\{ID, name\}$ 不是候选码, 因为它的子集 $\{ID\}$ 已经是超码了。候选码在一个关系中可能有多个。
- **主码 (Primary Key)**: 被数据库设计者选中的, 用于在关系中唯一标识元组的**候选码**。每个关系只能有一个主码。主码的值不能为 NULL。
- **外码 (Foreign Key)**: 一个关系中的属性 (或属性集), 它在另一个关系中是主码。外码是实现表与表之间**关联**的桥梁。
 - **参照完整性约束 (Referential Integrity Constraint)**: 要求外码的值必须是其参照关系中某个元组的主码值, 或者为 NULL。这保证了关系之间引用的有效性。

2.3. 关系代数 (Relational Algebra)

关系代数是一种**过程化查询语言 (procedural query language)**。它包含一系列运算, 这些运算以一个或多个关系为输入, 生成一个新的关系作为输出。

2.3.1. 基本运算 (Fundamental Operations)

- **选择 (Select)**: 选取关系中**满足给定条件的元组 (行)**。
 - 语法: $\sigma_{predicate}(r)$
 - 例子: 查询物理系 (Physics) 的所有教师。 $\sigma_{dept_name="Physics"}(instructor)$
- **投影 (Project)**: 选取关系中的**特定属性 (列)**, 并去掉重复的行。
 - 语法: $\Pi_{A1,A2,\dots,A_n}(r)$
 - 例子: 查询所有教师的 ID, 姓名和薪水。 $\Pi_{ID,name,salary}(instructor)$
- **并 (Union)**: 合并两个关系的所有元组, 并去掉重复项。两个关系必须**类型相容** (属性数量相同, 对应属性域相同)。
 - 语法: $r \cup s$
 - 例子: 找出在2017年秋季学期或2018年春季学期开设的所有课程。
$$\Pi_{course_id}(\sigma_{semester="Fall" \wedge year=2017}(section)) \cup$$
$$\Pi_{course_id}(\sigma_{semester="Spring" \wedge year=2018}(section))$$
- **差 (Set Difference)**: 找出在第一个关系中但不在第二个关系中的元组。两个关系必须类型相容。
 - 语法: $r - s$

- 例子：找出在2017年秋季学期开设，但不在2018年春季学期开设的课程。

$\Pi_{course_id}(\sigma_{semester="Fall" \wedge year=2017}(section)) -$

$\Pi_{course_id}(\sigma_{semester="Spring" \wedge year=2018}(section))$

- **笛卡尔积 (Cartesian Product)**：将两个关系的每一个元组进行组合。
 - 语法： $r \times s$
 - 它本身不常用，通常与 `Select` 结合使用，形成 `Join` (连接) 的效果。
- **更名 (Rename)**：为关系或其属性赋予新的名字。
 - 语法： $\rho_x(E)$ 或 $\rho_{x(A_1, \dots, A_n)}(E)$
 - 这在需要对一个关系进行自连接或需要简化复杂表达式时非常有用。

2.3.2. 附加及扩展运算 (Additional & Extended Operations)

- **交 (Set Intersection)**：找出同时存在于两个关系中的元组。可以用差运算表示： $r \cap s = r - (r - s)$ 。
- **自然连接 (Natural Join - $\bowtie$)**：这是一个非常重要的运算。它在两个关系的**公共属性**上进行匹配，并将匹配成功的元组合并成一个新的元组。
 - 语法： $r \bowtie s$
 - 例子：查询所有教师的姓名以及他们所教课程的 ID。 `instructor  $\bowtie$  teaches`
- **外连接 (Outer Join)**：自然连接会丢弃那些在另一个关系中找不到匹配的元组。外连接则会保留它们，并在缺失的属性上填入 `NULL`。
 - **左外连接 (Left Outer Join - $\bowtie\leftarrow$)**：保留左侧关系的所有元组。
 - **右外连接 (Right Outer Join - $\rightarrow\bowtie$)**：保留右侧关系的所有元组。
 - **全外连接 (Full Outer Join - $\bowtie\leftrightarrow$)**：保留两侧关系的所有元组。

3. 第三章：SQL (Chapter 3: SQL)

SQL 是用于管理关系数据库管理系统 (RDBMS) 的标准语言。它主要分为几个部分：数据定义语言 (DDL)、数据操纵语言 (DML)、数据控制语言 (DCL) 和事务控制语言 (TCL)。本章我们主要关注 DDL 和 DML。

3.1. 数据定义语言 (Data Definition Language - DDL)

DDL 用于定义和管理数据库的结构，比如创建或删除表。

- **创建表 (CREATE TABLE)**

我们使用 `CREATE TABLE` 命令来新建一个表，同时定义它的列名、数据类型以及各种约束。

- **常用数据类型 (Common Data Types)**：

- `char(n)` : 固定长度的字符串。
- `varchar(n)` : 可变长度的字符串。
- `int` : 整数类型。
- `numeric(p, d)` : 定点数, `p` 是总位数, `d` 是小数位数。
- `real, double precision` : 浮点数。
- `date` : 日期。
- **完整性约束 (Integrity Constraints):**
 - `primary key (A)` : 定义 `A` 为主码。
 - `foreign key (B) references r(A)` : 定义 `B` 为外码, 参照关系 `r` 的主码 `A`。
 - `not null` : 约束该列的值不能为空。
- **例子:** 创建一个 `department` (系) 表。

```
CREATE TABLE department (
    dept_name varchar(20),
    building varchar(15),
    budget numeric(12, 2),
    primary key (dept_name)
);
```

3.2. 基本查询结构 (Basic Query Structure)

查询是 SQL 最核心的功能, 基本结构由 `SELECT`, `FROM`, `WHERE` 三个子句组成。

- **SELECT 子句:** 列出查询结果中需要包含的列。
 - 使用 `DISTINCT` 去除结果中的重复行。
 - 使用 `*` 代表所有列。
 - 可以在 `SELECT` 中使用算术表达式, 如 `salary / 12`。
- **FROM 子句:** 指定查询操作所涉及的表。如果涉及多个表, `FROM` 子句会计算这些表的笛卡尔积。
- **WHERE 子句:** 设置查询条件, 过滤 `FROM` 子句结果中不满足条件的行。
 - 可以使用 `and`, `or`, `not` 等逻辑操作符组合条件。
- **例子:** 查询物理系 (Physics) 教师的姓名和工资。

```
SELECT name, salary
FROM instructor
WHERE dept_name = 'Physics';
```

- **重命名 (AS):** 可以使用 `AS` 关键字为表或列提供一个临时的别名, 使查询更具可读性。

```
SELECT name AS instructor_name, course_id
FROM instructor AS T, teaches AS S
WHERE T.ID = S.ID;
```

3.3. 附加的基本运算

- **字符串匹配 (LIKE)**: 用于在 WHERE 子句中进行模糊查询。
 - %: 匹配任意子串。
 - _: 匹配任意一个字符。
 - 例子: 查询所有姓名中包含 "dar" 的教师。

```
SELECT name
FROM instructor
WHERE name LIKE '%dar%';
```

- **结果排序 (ORDER BY)**: 对查询结果进行排序。
 - ASC: 升序 (默认)。
 - DESC: 降序。
 - 例子: 按薪水降序查询所有教师。

```
SELECT *
FROM instructor
ORDER BY salary DESC;
```

- **集合运算 (Set Operations)**:
 - UNION: 合并两个查询的结果，并去除重复行 (UNION ALL 则保留重复行)。
 - INTERSECT: 返回两个查询结果的交集。
 - EXCEPT: 返回在第一个查询结果中但不在第二个查询结果中的行。

3.4. 聚合函数 (Aggregate Functions)

聚合函数对一组值进行计算，并返回单个值。

- **常用函数**:
 - AVG(): 平均值。
 - MIN(): 最小值。
 - MAX(): 最大值。
 - SUM(): 总和。
 - COUNT(): 计数。
- **分组 (GROUP BY)**: 将具有相同值的行分组，以便聚合函数可以对每个组进行计算。

- 例子：查询每个系的平均工资。

```
SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor
GROUP BY dept_name;
```

- **分组过滤 (HAVING)**：在 GROUP BY 分组后，对分组结果进行条件过滤。
 - **WHERE 和 HAVING 的区别**：WHERE 在分组前过滤行，HAVING 在分组后过滤组。
 - 例子：查询平均工资超过 42000 美元的系。

```
SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) > 42000;
```

3.5. 嵌套子查询 (Nested Subqueries)

子查询是嵌套在另一个 SQL 查询的 WHERE 或 HAVING 等子句中的 SELECT-FROM-WHERE 表达式。

- **IN / NOT IN**：测试一个值是否存在于子查询返回的结果集中。
 - 例子：找出在2017年秋季和2018年春季都开课的课程。

```
SELECT DISTINCT course_id
FROM section
WHERE semester = 'Fall' AND year = 2017 AND
      course_id IN (SELECT course_id
                    FROM section
                    WHERE semester = 'Spring' AND year = 2018);
```

- **EXISTS / NOT EXISTS**：测试子查询是否返回任何行。
 - EXISTS 通常比 IN 效率更高，因为它一旦找到匹配行就会停止搜索。
- **WITH 子句**：允许你定义一个临时的、在主查询中可以引用的命名关系。这可以使复杂查询的结构更清晰。
 - 例子：查询预算最高的系。

```
WITH max_budget(value) AS (
    SELECT MAX(budget)
    FROM department
)
SELECT dept_name
FROM department, max_budget
WHERE department.budget = max_budget.value;
```

4. 第四章：中级 SQL (Chapter 4: Intermediate SQL)

4.1. 连接表达式 (Join Expressions)

在上一章，我们通过从 `FROM` 子句中列出多个表来实现连接。SQL 提供了更明确、更具可读性的 `JOIN` 语法。

- **内连接 (Inner Join)**：返回两个表中连接字段相匹配的行。 `INNER JOIN` 在功能上等同于我们在 `WHERE` 子句中指定连接条件的写法。

```
-- 查询教师及其所在系的建筑名称
SELECT name, building
FROM instructor INNER JOIN department ON instructor.dept_name = department.dept_name;
```

- **外连接 (Outer Join)**：外连接是内连接的扩展。如果某表的行在另一个表中没有匹配，它同样可以出现在结果中，其对应另一个表的列将显示为 `NULL`。
 - `LEFT OUTER JOIN`：保留左表的所有行。
 - `RIGHT OUTER JOIN`：保留右表的所有行。
 - `FULL OUTER JOIN`：保留两个表的所有行。
 - 例子：查询所有教师及其教授的课程，即使某些教师没有教任何课，也要将他们列出。

```
SELECT T.name, S.course_id
FROM instructor AS T LEFT OUTER JOIN teaches AS S ON T.ID = S.ID;
```

- **连接条件 (ON vs. USING)**
 - `ON` 子句：允许我们指定任意的连接条件。
 - `USING` 子句：当两个表中用于连接的列同名时，可以使用 `USING` 来简化语法。

```
-- 使用 USING 的等价查询
-- 假设 instructor 和 teaches 中都有 ID 列
SELECT name, course_id
FROM instructor JOIN teaches USING(ID);
```

4.2. 视图 (Views)

视图 (View) 本质上是一个**虚拟表 (virtual table)**，它的内容由一个查询定义。视图本身不存储数据，数据仍然存储在它所引用的基表 (base table) 中。

- 视图的优点：

- **简化复杂查询**：将复杂的连接或聚合操作封装在一个视图中。
- **增强安全性**：可以只向用户暴露视图，隐藏基表的某些列或行，从而限制用户的数据访问。
- **创建视图 (CREATE VIEW)**
 - 例子：创建一个视图，显示每个系的总工资。

```
CREATE VIEW department_total_salary(dept_name, total_salary) AS
SELECT dept_name, SUM(salary)
FROM instructor
GROUP BY dept_name;
```

- 之后，我们可以像查询普通表一样查询这个视图： `SELECT * FROM department_total_salary;`
- **视图的可更新性 (Updatability of Views)**：并非所有视图都是可更新的。如果视图的定义涉及聚合、 `GROUP BY` 或 `DISTINCT` 等操作，通常不能通过视图来更新（ `INSERT` , `UPDATE` , `DELETE` ）基表。

4.3. 事务 (Transactions)

我们在第一章提过的 ACID 特性，在 SQL 中通过**事务控制语言 (Transaction Control Language)** 来实现。一个事务是一系列操作的集合，它们要么全部成功，要么全部失败。

- **提交事务 (COMMIT)**：当一个事务成功完成后，使用 `COMMIT` 命令可以将其所有修改永久地保存到数据库中。

```
COMMIT WORK;
```

- **回滚事务 (ROLLBACK)**：如果在事务过程中发生错误，或你希望撤销所做的更改，可以使用 `ROLLBACK` 命令将数据库恢复到事务开始前的状态。

```
ROLLBACK WORK;
```

4.4. 完整性约束 (Integrity Constraints)

除了主码和外码，SQL 还提供了其他约束来保证数据的正确性。

- **CHECK 约束**：为一个属性或一组属性指定一个必须为真的条件。
 - 例子：在 `department` 表中确保预算 `budget` 必须为正数。

```
CREATE TABLE department (  
    dept_name varchar(20) PRIMARY KEY,  
    building varchar(15),  
    budget numeric(12, 2) CHECK (budget > 0)  
);
```

- **参照完整性中的级联操作 (Cascading Actions in Referential Integrity)**: 当删除或更新被外码引用的主码时, 可以指定数据库自动执行的操作。
 - ON DELETE CASCADE: 当主表中的记录被删除时, 自动删除子表中所有引用它的记录。
 - ON DELETE SET NULL: 当主表中的记录被删除时, 自动将子表中引用它的外码字段设置为 NULL。

4.5. 授权与安全 (Authorization and Security)

数据库的安全性通过授权机制来管理, 即控制不同用户可以对哪些数据执行哪些操作。

- **权限 (Privileges)**: 包括 SELECT, INSERT, UPDATE, DELETE 等。
- **授权 (GRANT)**: 授予用户或角色特定的权限。
 - 例子: 授予用户 Amit 查询 instructor 表的权限。

```
GRANT SELECT ON instructor TO Amit;
```

- **撤销权限 (REVOKE)**: 收回之前授予的权限。
 - 例子: 从用户 Amit 那里收回查询 instructor 表的权限。

```
REVOKE SELECT ON instructor FROM Amit;
```

- **角色 (ROLE)**: 角色是一组权限的集合。我们可以将权限授予一个角色, 然后将这个角色赋予多个用户, 从而极大地简化权限管理。
 - CREATE ROLE instructor_role;
 - GRANT SELECT ON takes TO instructor_role;
 - GRANT instructor_role TO Amit;

5. 第六章：关系数据库设计 (Chapter 6: Relational Database Design)

5.1. 优良设计的特点 (Features of Good Relational Design)

一个好的数据库设计应该避免 **数据冗余 (data redundancy)**（即同样的信息被存储多次）以及由冗余所导致的 **更新异常 (update anomalies)**。

- **更新异常 (Update Anomalies)**

- **插入异常 (Insertion Anomaly)**：当我们想插入某个实体的信息时，却因为缺少另一个实体的信息而无法插入。例如，一个银行网点刚建立，还没有任何贷款业务，我们就无法将这个网点的信息存入一张包含贷款信息的表中。
- **删除异常 (Deletion Anomaly)**：当我们删除一条记录时，可能会导致我们想保留的其他信息也一并丢失。例如，删除了某个网点的最后一笔贷款记录，可能会把这个网点本身的信息（如地址、资产）也从数据库中抹去。
- **修改异常 (Modification Anomaly)**：当我们要修改一个信息时，却需要更新多行记录。例如，要修改某个网点的地址，就需要找到所有与该网点相关的贷款记录并逐一修改。

- **分解 (Decomposition)**

解决这些问题的核心方法就是 **分解 (decompose)**，即将一个存在问题的“大表”拆分成多个结构良好的“小表”。

5.2. 函数依赖 (Functional Dependencies - FDs)

为了能够正确地进行分解，我们需要一种形式化的工具来描述属性之间的约束关系，这个工具就是 **函数依赖 (Functional Dependency)**。

- **概念与定义 (Concept and Definition)**

如果在一个表中，属性集 α 的值能够唯一地 **决定 (determine)** 属性集 β 的值，那么我们就说存在一个函数依赖，记作 $\alpha \rightarrow \beta$ 。

简单来说就是：只要知道了 α 的值， β 的值也就确定了。对于表中的任意两行，如果它们的 α 值相同，那么它们的 β 值也**必须**相同。

- **例子**：在一个 `student`（学生）表中，存在函数依赖 $ID \rightarrow name$ 。只要我们知道了学生的学号 (ID)，我们就能唯一地确定他的姓名 (name)。

- **函数依赖与码 (FDs and Keys)**

“码”的概念是函数依赖的一个特例。如果属性集 K 是表 R 的一个 **超码 (superkey)**，那么函数依赖 $K \rightarrow R$ 必然成立。

5.3. 范式 (Normal Forms)

范式 (Normal Forms) 是一系列用于衡量表结构优良程度的规则或标准，其目的在于消除更新异常。

- **Boyce-Codd 范式 (Boyce-Codd Normal Form - BCNF)**

BCNF 是一个非常高的规范化标准。一个表属于 BCNF 的条件是：对于表中存在的每一个非平凡的函数依赖 $\alpha \rightarrow \beta$ ， α 都必须是该表的超码。(平凡的函数依赖指的是 β 是 α 的子集的情况)。

- **经验法则：**在一个 BCNF 的表中，所有函数依赖的决定因素（左边的 α ）都必须是超码。这可以从根本上杜绝我们之前讨论的各种异常。

- **第三范式 (Third Normal Form - 3NF)**

3NF 是一个比 BCNF 稍微宽松一些的标准。一个表属于 3NF 的条件是：对于每一个非平凡的函数依赖 $\alpha \rightarrow \beta$ ，至少满足以下两项之一：

- i. α 是一个超码。(与 BCNF 规则相同)。
- ii. 每一个在 β 中但不在 α 中的属性，都包含在 R 的某个 **候选码 (candidate key)** 中。(这样的属性也称为**主属性**)。

5.4. BCNF 与 3NF 的比较 (Comparison of BCNF and 3NF)

虽然 BCNF 更严格，通常被认为是理想形态，但在实际应用中，3NF 具有一个非常重要的优势。

- **分解的目标 (Goals of Decomposition)**

- **无损连接 (Lossless Join)：**当我们将一个表分解成多个小表后，必须保证能够通过自然连接将它们恢复成原始信息，而不会产生任何多余的、错误的记录。
- **依赖保持 (Dependency Preservation)：**分解之后，原表中所有的函数依赖都应该能在某个新的小表中被单独检查，而无需连接多个表。

- **BCNF vs. 3NF**

- 分解到 **BCNF** 总是无损连接的，但不一定能保持所有的依赖。
- 分解到 **3NF** 既能保证无损连接，又能保证依赖保持。

正因如此，在实际工程中，有时一个 3NF 的设计方案会是比纯粹 BCNF 方案更好的折中选择。

6. 第七章：存储与文件结构 (Chapter 7: Storage and File Structure)

6.1. 物理存储介质 (Physical Storage Media)

数据库系统使用不同类型的存储介质来存放数据，它们构成了一个**存储层次结构 (Storage Hierarchy)**。

- **存储层次结构:**
 - **易失性存储 (Volatile Storage):** 速度最快但断电后数据会丢失。例如：高速缓存 (cache) 和主存 (main memory/RAM)。
 - **非易失性存储 (Non-volatile Storage):** 断电后数据依然保留，速度比主存慢。例如：闪存 (Flash Memory, SSD) 和磁盘 (Magnetic Disk, HDD)。这是数据库主体数据存放的地方。
 - **三级存储 (Tertiary Storage):** 用于归档和备份，速度最慢，成本最低。例如：磁带 (magnetic tape)。

- **磁盘 (Magnetic Disks)**

磁盘是传统数据库最主要的存储介质。数据以**块 (block)** 为单位在磁盘和主存之间传输。访问磁盘数据的时间主要由三部分构成：

- i. **寻道时间 (Seek Time):** 将磁头移动到目标磁道所需的时间。
- ii. **旋转延迟 (Rotational Latency):** 等待磁盘旋转，使目标扇区到达磁头下方的时间。
- iii. **传输时间 (Transfer Time):** 数据从磁盘传输到主存的时间。

其中，寻道时间和旋转延迟是主要瓶颈，因此数据库的设计目标之一就是**最小化磁盘 I/O 次数**。

6.2. 文件中的记录组织 (Organization of Records in Files)

数据库中的数据最终以**记录 (records)** 的形式存储在磁盘**块 (blocks)** 中。

- **定长记录 (Fixed-Length Records)**

当所有记录的长度都相同时，存储很简单：在块中一个接一个地线性排列即可。我们可以通过简单的算术运算快速定位第 i 条记录的位置。

- **变长记录与槽页结构 (Variable-Length Records and the Slotted-Page Structure)**

当记录的长度可变时（例如，包含 `varchar` 类型的字段），存储变得复杂。**槽页结构 (Slotted-Page Structure)** 是处理这种情况的一种高效方式。

- **块头 (Block Header):** 每个块的开头部分包含一个块头，用于存储控制信息。
- **槽 (Slots):** 块头中包含一个由多个“槽”组成的入口数组，每个槽存放对应记录在块内的**位置**和**长度**。
- **记录存储:** 记录从块的末尾向前“生长”。

- **优点:** 当记录需要移动或删除时，只需修改块头中的槽信息，而无需移动其他记录，从而提高了效率。

6.3. 文件组织方式 (File Organization)

一个文件由多个块组成，文件中记录的排列方式，即**文件组织 (File Organization)**，对数据访问效率有巨大影响。

- **堆文件组织 (Heap File Organization)**

记录可以被放置在文件的任何地方，只要该处有空间。

- **优点:** 插入非常快。
- **缺点:** 查询时必须线性扫描整个文件，速度极慢。

- **顺序文件组织 (Sequential File Organization)**

记录根据某个**搜索码 (search key)** 的值按顺序存储。

- **优点:** 对于基于搜索码的范围查询（例如，查询工资在 50000 到 60000 之间的所有教师）非常高效。
- **缺点:** 插入和删除操作很慢，因为可能需要移动大量记录来维持顺序。

- **散列文件组织 (Hashing File Organization)**

根据记录的某个或多个字段计算出一个**散列值 (hash value)**，这个值决定了记录应该被存放在哪个块中。

- **优点:** 对于基于散列字段的精确匹配查询（例如，`WHERE ID = '12345'`）非常快。
- **缺点:** 不支持范围查询。

- **集簇文件组织 (Clustering File Organization)**

将来自不同表但相互关联的记录存储在同一个块中。

- **例子:** 对于 `instructor` 表和 `department` 表，如果经常需要通过 `dept_name` 连接它们，我们可以将某个系的所有教师记录和该系的记录存储在物理上相邻的位置。
- **优点:** 极大地提升了连接 (Join) 操作的效率。

7. 第八章：索引与散列 (Chapter 8: Indexing and Hashing)

7.1. 索引的基本概念 (Basic Concepts of Indexing)

索引的原理类似于书籍的目录，它让我们无需阅读整本书就能快速定位到需要的内容。

- **有序索引 (Ordered Indices):** 索引项根据**搜索码 (search key)** 的值排序存储。

- **主索引 (Primary Index)**: 也称为**聚簇索引 (Clustering Index)**。其索引的搜索码与文件中记录的物理存储顺序一致。一个文件最多只能有一个主索引。
- **辅助索引 (Secondary Index)**: 也称为**非聚簇索引 (Non-clustering Index)**。其索引顺序与记录的物理存储顺序无关。一个表可以有多个辅助索引。
- **稠密索引 (Dense Index)**: 文件中**每一条**记录的搜索码值, 在索引中都有一个对应的条目。**辅助索引必须是稠密索引**。
- **稀疏索引 (Sparse Index)**: 只为文件中**一部分**记录的搜索码值建立索引条目 (通常是每个磁盘块的第一条记录)。**稀疏索引只能建立在主索引上**。

7.2. B+ 树索引 (B+-Tree Index Files)

B+ 树 (B+-Tree) 是迄今为止数据库中使用最广泛、最重要的索引结构。它是一种自平衡的多路搜索树。

- **B+ 树的结构**
 - **内部节点 (Internal Nodes)**: 仅包含用于导航的键值和指向下一层节点的指针, 不存储指向真实数据的指针。
 - **叶子节点 (Leaf Nodes)**: 包含索引的搜索码值以及指向磁盘上实际数据记录的指针。
 - **顺序集 (Sequence Set)**: 所有的叶子节点通过指针相互链接, 形成一个有序的链表。
- **B+ 树的特点**
 - **平衡性 (Balanced)**: 从根节点到任意一个叶子节点的路径长度都是相同的。这保证了查询性能的稳定, 时间复杂度为 $O(\log_f N)$, 其中 f 是树的扇出 (fanout), N 是记录数。
 - **高扇出 (High Fanout)**: 由于内部节点不存数据指针, 可以存放更多的键值, 使得树的扇出非常大, 从而树的高度非常低 (通常3-4层), 极大地减少了磁盘 I/O 次数。
 - **范围查询友好**: 叶子节点组成的有序链表使得范围查询 (如 `WHERE age > 20 AND age < 30`) 极为高效。
- **B+ 树 vs. B 树**

B 树与 B+ 树类似, 但 B 树的**内部节点也存储指向数据的指针**。这导致其扇出较小, 树的高度可能更高。在数据库中, B+ 树因其更高的扇出和对范围查询的优化而更受青睐。

7.3. 散列索引 (Hash Indices)

散列是另一种实现快速查找的索引技术。

- **静态散列 (Static Hashing)**
 - 它将文件空间划分为固定的 **桶 (buckets)**。
 - 通过一个**散列函数 (hash function)** 将搜索码映射到某个桶。

- **缺点:** 如果文件大小变化剧烈,可能会导致大量**桶溢出 (bucket overflow)**,形成长长的溢出链,严重影响性能。

- **动态散列 (Dynamic Hashing)**

为了解决静态散列的问题,人们提出了动态散列方案,如**可扩展散列 (Extendable Hashing)**。它允许散列表根据数据量的增减来动态地扩大或缩小,从而保持良好的性能。

7.4. 索引选择的权衡 (Trade-offs in Index Selection)

虽然索引能加速查询,但它不是免费的午餐。

- **空间开销:** 每个索引都需要额外的磁盘空间来存储。
- **维护开销:** 每当对表进行插入、删除、修改操作时,相关的索引也必须被更新,这会降低写操作的性能。
- 因此,创建索引需要在**查询性能的提升**和**写操作性能的下降**之间做出权衡。通常只在经常被用作查询条件的列上创建索引。

8. 第九章：查询处理 (Chapter 9: Query Processing) 仅作补充

8.1. 查询处理概述 (Overview of Query Processing)

当用户提交一条 SQL 查询时,数据库系统并不会立即执行它。相反,它会经历一个复杂的处理流程,以找出最高效的执行方式。

- **查询处理的基本步骤**
 - i. **解析与转换 (Parsing and Translation):** 系统首先检查 SQL 查询的语法,然后将其翻译成内部表示形式,通常是一棵**关系代数表达式树**。
 - ii. **优化 (Optimization):** 这是最关键的一步。查询优化器会生成多个逻辑上等价但执行效率可能天差地别的**查询执行计划 (Query Execution Plan)**,并利用统计信息估算每个计划的成本,最终选出成本最低的一个。
 - iii. **求值 (Evaluation):** 查询执行引擎接收优化器给出的最佳计划,并执行它,最终返回查询结果。
- **查询代价的衡量 (Measures of Query Cost)**

查询的代价主要包括 CPU 时间、磁盘 I/O、网络通信等。在集中式数据库中,**磁盘 I/O 的成本是主要矛盾**,因此优化的核心目标通常是**最小化磁盘块的读写次数**。

8.2. 查询优化 (Query Optimization)

查询优化的本质是在众多等价的执行方案中选择最优的一个。

- **关系代数表达式等价变换**

优化器利用关系代数的等价规则来重写查询。一个最著名、最有效的启发式规则是：**尽可能早地执行选择 (Selection) 和投影 (Projection) 操作。**

- **例如：** $\sigma_{\text{condition}}(R \bowtie S)$ 远比 $(\sigma_{\text{condition}}(R)) \bowtie S$ 要低效（假设 `condition` 只涉及 `R` 的属性）。先做选择可以大大减少参与连接操作的元组数量，从而降低中间结果的大小和后续操作的成本。

8.3. 算法选择 (Algorithm Selection)

对于关系代数中的每一个操作（如选择、连接），都有多种实现算法可供选择。

- **选择操作的实现 (Algorithms for Selection)**

- **A1 (线性扫描):** 暴力扫描整个文件，适用于任何情况，但效率最低。
- **A2 (二分查找):** 如果文件按选择属性有序，则可以使用二分查找，效率很高。
- **使用索引:** 如果选择属性上有索引（如 B+ 树或散列索引），则可以利用索引快速定位，这是最高效的方式。

- **连接操作的实现 (Algorithms for Join)**

连接操作通常是查询中最耗时的部分，其算法选择至关重要。

- **嵌套循环连接 (Nested-Loop Join):** 最简单直观的算法，像两层 for 循环一样，用外层关系的每一行去匹配内层关系的所有行。对于大数据集，其性能非常差。
- **块嵌套循环连接 (Block Nested-Loop Join):** 对朴素嵌套循环的改进，每次读取外层关系的“一块”而不是一行，大大减少了扫描内层关系的次数。
- **归并连接 (Merge Join):** 如果两个关系都已在连接属性上排好序，则此算法非常高效。它只需要同时扫描两个关系一次即可完成连接。
- **散列连接 (Hash Join):** 将两个关系都使用同一个散列函数按连接属性划分到不同的桶（分区）中，然后分别对每一对相应的桶进行连接。对于大型、无序关系的等值连接，此算法通常是最高效的。

8.4. 表达式求值 (Evaluation of Expressions)

当优化器生成了一个由多个操作组成的计划树后，执行引擎需要具体地执行它。

- **物化 (Materialization)**

这是一种“一步一脚印”的方式。引擎执行树上的一个操作，将其结果**完整地写入一个临时的中间表**存到磁盘上，然后再由上一个操作读取这个临时表。

- **缺点:** 产生了大量的磁盘 I/O，成本很高。

- **流水线 (Pipelining)**

这是一种更高效的方式。一个操作的结果**不写入磁盘，而是直接作为输入传递给流水线中的下一个操作**。这就像工厂的流水线一样，多个操作可以同时进行。

- **优点:** 极大地节省了磁盘 I/O，是现代数据库执行引擎的首选策略。

9. 第十章：事务 (Chapter 10: Transaction)

9.1. 事务的概念 (Transaction Concept)

- **什么是事务？**

事务 (Transaction) 是访问并可能更新数据库中各种数据项的一个**程序执行单元**。它是一个不可分割的**逻辑工作单元**，即使它可能由多个独立的操作组成。

- **经典例子:** 银行转账。从账户 A 转 100 元到账户 B，这个操作包含两个步骤：

- a. 从 A 的余额中减去 100 元。

- b. 向 B 的余额中增加 100 元。

这两个步骤必须被捆绑在一起，要么都成功，要么都失败。我们绝不允许只扣款不存款的情况发生。这个“转账”操作就是一个事务。

- **事务的目标**

事务的主要目标是，在即使出现系统故障（如断电、系统崩溃）或并发访问的情况下，依然能够保持数据库的**一致性 (Consistency)**。

9.2. ACID 特性 (The ACID Properties)

为了实现上述目标，事务必须严格遵循四个基本特性，合称为 **ACID**。

1. **原子性 (Atomicity)**

这是“全有或全无 (all-or-nothing)”的特性。它要求事务中的所有操作要么**全部执行**，要么就**一个都不执行**。如果事务在执行过程中任何一点失败，系统必须能撤销（回滚）该事务所做的所有修改，将数据库恢复到事务开始前的状态。

- **对应例子:** 银行转账必须是原子的。不能只扣了 A 的款，而 B 的款没到账。

2. **一致性 (Consistency)**

它保证事务必须使数据库从一个**一致的状态转变到另一个一致的状态**。事务的执行不能破坏数据库的任何完整性约束（如主码、外码、CHECK约束）。

- **对应例子:** 在转账前后，A 和 B 账户的总金额应该是不变的。如果转账破坏了这个平衡，就是违反了一致性。

3. **隔离性 (Isolation)**

在有多多个事务并发执行时，一个事务的执行不应被其他事务干扰。也就是说，每个事务都感觉不到

系统中有其他事务在并发地执行，就好像这些事务是**串行 (serially)** 执行的一样。

- **对应例子:** 当我正在从 A 转账到 B 的过程中，另一个并发的查询操作不应该看到 A 的钱已经扣了，但 B 的钱还没到账的这种“中间状态”。

4. 持久性 (Durability)

一旦一个事务被成功**提交 (commit)**，它对数据库的改变就必须是**永久性的**。即使在提交后系统立即发生崩溃，数据库也必须能够通过恢复机制（如重做日志）来确保这些改变不会丢失。

- **对应例子:** 一旦银行系统提示你转账成功，那么这次转账的结果就必须被永久保存下来，无论之后发生任何故障。

9.3. 事务的状态 (Transaction State)

一个事务在其生命周期中会经历以下几种状态：

- **活动 (Active):** 事务的初始状态，表示事务正在执行中。
- **部分提交 (Partially Committed):** 当事务的最后一条语句执行完毕后，进入此状态。此时，事务已经完成了所有计算，但数据还未永久写入磁盘。
- **失败 (Failed):** 当事务在执行过程中发现任何错误，无法正常继续时，进入此状态。
- **中止 (Aborted):** 在事务失败后，系统会**回滚 (rollback)** 该事务所做的所有修改，数据库恢复到事务开始前的状态。完成回滚后，事务进入中止状态。
- **提交 (Committed):** 当事务成功完成所有操作（即处于部分提交状态后），系统会将其修改永久地写入数据库，此时事务进入提交状态。